

1.SMART UNIVERSITY PAYROLL MANAGEMENT SYSTEM:

 **SIMATS**
ENGINEERING

SIMATS Online Programming Lab
JAVA PROGRAMMING

 SANGPU VENKATA PRAVEEN
192373013RD

Your Code

```
1 import java.util.Scanner;
2 class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int n = Integer.parseInt(s.nextLine());
6         Employee[] emp = new Employee[n];
7         for (int i = 0; i < n; i++) {
8             String type = s.nextLine();
9             int id = Integer.parseInt(s.nextLine());
10            String name = s.nextLine();
11            String dept = s.nextLine();
12            double basicSalary = Double.parseDouble(s.nextLine());
13            if (type.equalsIgnoreCase("TeachingFaculty")) {
14                int techingHour = Integer.parseInt(s.nextLine());
15                double incentivePerHour = Double.parseDouble(s.nextLine());
16
17                emp[i] = new TeachingFaculty(id, name, dept, basicSalary, techingHour, incentivePerHour);
18            } else if (type.equalsIgnoreCase("TechnicalStaff")) {
19                int overtimeHour = Integer.parseInt(s.nextLine());
20                double overtimeRate = Double.parseDouble(s.nextLine());
21
22                emp[i] = new TechnicalStaff(id, name, dept, basicSalary, overtimeHour, overtimeRate);
23            } else if (type.equalsIgnoreCase("VisitingFaculty")) {
24                int lectureHandled = Integer.parseInt(s.nextLine());
25                double paymentPerHour = Double.parseDouble(s.nextLine());
26                emp[i] = new VisitingFaculty(id, name, dept, basicSalary, lectureHandled, paymentPerHour);
27            }
28        }
29        int searchID = Integer.parseInt(s.nextLine());
30        System.out.println("Employee Details");
31        System.out.println();
32        double total = 0;
33        double maxSalary = -1;
34        Employee highest = null;
35        for (Employee e : emp) {
36            e.display();
37            double salary = e.calculateSalary();
38            System.out.println("Monthly Salary : " + salary);
39            System.out.println();
40            total += salary;
41            if (salary > maxSalary) {
42                maxSalary = salary;
43                highest = e;
44            }
45        }
46        System.out.println("Highest Paid Employee");
47        System.out.println("Employee ID : " + highest.getId());
48        System.out.println("Salary : " + highest.calculateSalary());
49        System.out.println();
50        System.out.println("Average Salary");
51        System.out.printf("%.2f\n", total / n);
52        System.out.println();
53    }
54 }
```

Input

3

TeachingFaculty

101

Output

 **SIMATS**
ENGINEERING

SIMATS Online Programming Lab
JAVA PROGRAMMING

 SANGPU VENKATA PRAVEEN
192373013RD

```
26
27         emp[i] = new VisitingFaculty(id, name, dept, basicSalary, lectureHandled, paymentPerHour);
28     }
29 }
30
31 int searchID = Integer.parseInt(s.nextLine());
32 System.out.println("Employee Details");
33 System.out.println();
34 double total = 0;
35 double maxSalary = -1;
36 Employee highest = null;
37 for (Employee e : emp) {
38     e.display();
39     double salary = e.calculateSalary();
40     System.out.println("Monthly Salary : " + salary);
41     System.out.println();
42     total += salary;
43     if (salary > maxSalary) {
44         maxSalary = salary;
45         highest = e;
46     }
47 }
48 System.out.println("Highest Paid Employee");
49 System.out.println("Employee ID : " + highest.getId());
50 System.out.println("Salary : " + highest.calculateSalary());
51 System.out.println();
52 System.out.println("Average Salary");
53 System.out.printf("%.2f\n", total / n);
54 System.out.println();
55 }
```

```

54      System.out.println("");
55      boolean found = false;
56      for (Employee e : emp) {
57          if (e.getId() == searchID()) {
58              System.out.println("Employee Found");
59              System.out.println("Employee ID : " + e.getId());
60              System.out.println("Name : " + e.getName());
61              System.out.println("Salary : " + e.calculateSalary());
62              found = true;
63              break;
64          }
65      }
66      if (!found) {
67          System.out.println("Employee Not Found");
68      }
69      s.close();
70  }
71  }
72  class Employee {
73      private int empId;
74      private String empName;
75      private String department;
76      private double basicSalary;
77      Employee(int empId, String empName, String department, double basicSalary) {
78          this.empId = empId;
79          this.empName = empName;
80          this.department = department;
81          this.basicSalary = basicSalary;
82      }

```

```

83      }
84      public int getId() {
85          return empId;
86      }
87      public String getName() {
88          return empName;
89      }
90      public String getDepartment() {
91          return department;
92      }
93      public double getBasicSalary() {
94          return basicSalary;
95      }
96      public double calculateSalary() {
97          return basicSalary;
98      }
99      void display() {
100          System.out.println("Employee ID : " + empId);
101          System.out.println("Name : " + empName);
102          System.out.println("Department : " + department);
103      }
104  class TeachingFaculty extends Employee {
105      private int teachingHour;
106      private double incentivePerHour;
107      TeachingFaculty(int id, String name, String dept, double basicSalary, int teachingHour, double incentivePerHour) {
108          super(id, name, dept, basicSalary);
109          this.teachingHour = teachingHour;
110          this.incentivePerHour = incentivePerHour;

```

```

111     }
112     @Override
113     public double calculateSalary() {
114         return getBasicSalary() + (techingHour * incentivePerHour);
115     }
116 }
117 class TechnicalStaff extends Employee {
118     private int overtimeHour;
119     private double overtimeRate;
120     TechnicalStaff(int id, String name, String dept, double basicSalary) {
121         super(id, name, dept, basicSalary);
122         this.overtimeHour = overtimeHour;
123         this.overtimeRate = overtimeRate;
124     }
125     @Override
126     public double calculateSalary() {
127         return getBasicSalary() + (overtimeHour * overtimeRate);
128     }
129 }
130 class VisitingFaculty extends Employee {
131     private int lectureHandled;
132     private double paymentPerHour;
133     VisitingFaculty(int id, String name, String dept, double basicSalary) {
134         super(id, name, dept, basicSalary);
135         this.lectureHandled = lectureHandled;
136         this.paymentPerHour = paymentPerHour;
137     }
138     @Override
139     public double calculateSalary() {

```

2.SMART VEHICLE INSURANCE CLAIM SYSTEM:

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
JAVA PROGRAMMING

SANGEPU VENKATA PRAVEEN
192373013RD

```
1 import java.util.*;
2 public class Main{
3     static class Vehicle {
4         protected String vehicleNumber;
5         protected String ownerName;
6         protected double insuredAmount;
7         Vehicle(String vehicleNumber, String ownerName, double insuredAmount) {
8             this.vehicleNumber = vehicleNumber;
9             this.ownerName = ownerName;
10            this.insuredAmount = insuredAmount;
11        }
12        public String getVehicleNumber() {
13            return vehicleNumber;
14        }
15        public String getOwnerName() {
16            return ownerName;
17        }
18        public double calculateClaimAmount() {
19            return 0;
20        }
21    }
22    static class Car extends Vehicle {
23        Car(String vehicleNumber, String ownerName, double insuredAmount) {
24            super(vehicleNumber, ownerName, insuredAmount);
25        }
26        @Override
27        public double calculateClaimAmount() {
28            return insuredAmount * 0.80;
29        }
30    }
31}
```

3
Car
TN10AB1234Output
Vehicle : TN10AB1234
Claim Amount : 640000.0
Vehicle : TN09XY5678
Claim Amount : 60000.0

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
JAVA PROGRAMMING

SANGEPU VENKATA PRAVEEN
192373013RD

```
30 }
31 static class Bike extends Vehicle {
32     Bike(String vehicleNumber, String ownerName, double insuredAmount) {
33         super(vehicleNumber, ownerName, insuredAmount);
34     }
35     @Override
36     public double calculateClaimAmount() {
37         return insuredAmount * 0.60;
38     }
39 }
40 static class Truck extends Vehicle {
41     Truck(String vehicleNumber, String ownerName, double insuredAmount) {
42         super(vehicleNumber, ownerName, insuredAmount);
43     }
44     @Override
45     public double calculateClaimAmount() {
46         return insuredAmount * 0.90;
47     }
48 }
49
50 public static void main(String[] args){
51     Scanner sc = new Scanner(System.in);
52     int n = sc.nextInt();
53     Vehicle[] vehicles = new Vehicle[n];
54     for (int i = 0; i < n; i++) {
55         String type = sc.next();
56         String number = sc.next();
57         String owner = sc.next();
58         double amount = sc.nextDouble();
59     }
60 }
```

```

58         double amount = sc.nextDouble();
59         if (type.equalsIgnoreCase("Car"))
60             vehicles[i] = new Car(number, owner, amount);
61         else if (type.equalsIgnoreCase("Bike"))
62             vehicles[i] = new Bike(number, owner, amount);
63         else
64             vehicles[i] = new Truck(number, owner, amount);
65     }
66     String search = sc.next().trim();
67     Vehicle highest = vehicles[0];
68     for (Vehicle v : vehicles) {
69         System.out.println("Vehicle : " + v.getVehicleNumber());
70         System.out.println("Claim Amount : " + v.calculateClaimAmount());
71         System.out.println();
72         if (v.calculateClaimAmount() > highest.calculateClaimAmount())
73             highest = v;
74     }
75     System.out.println("Highest Claim Vehicle");
76     System.out.println(highest.getVehicleNumber());
77     System.out.println(highest.calculateClaimAmount());
78     System.out.println();
79     boolean found = false;
80     for (Vehicle v : vehicles) {
81         if (v.getVehicleNumber().trim().equalsIgnoreCase(search))
82             found = true;
83         System.out.println("Vehicle Found");
84         System.out.println("Owner : " + v.getOwnerName());
85     }

```

```

74     }
75     System.out.println("Highest Claim Vehicle");
76     System.out.println(highest.getVehicleNumber());
77     System.out.println(highest.calculateClaimAmount());
78     System.out.println();
79     boolean found = false;
80     for (Vehicle v : vehicles) {
81         if (v.getVehicleNumber().trim().equalsIgnoreCase(search))
82             found = true;
83         System.out.println("Vehicle Found");
84         System.out.println("Owner : " + v.getOwnerName());
85         System.out.println("Claim Amount : " + v.calculateClaimAmount());
86         break;
87     }
88     if (!found)
89         System.out.println("Vehicle Not Found");
90     sc.close();
91 }
92 }

```